COSC 369 Software Engineering I

Faculty Teaching and Office Hours Schedule Generator

Project report

Modular C application with a static library

Eldana Adinew

September 15 2026

1 Project Purpose and Result

This project implements a command-line system in C that reads one class-schedule file and one faculty-information file. It validates the records, creates each professor's teaching schedule, selects non-conflicting office-hour blocks, and writes text and CSV results. A Python script can convert the C-generated CSV files into a formatted Word document that follows the supplied sample's class-schedule and office-hours layout.

The completed sample processes three courses for one professor. It produces four office-hour blocks totaling 8.75 hours and reproduces the sample times without embedding those records in the source code. Changing the two input files changes the output without requiring recompilation.

Requirements addressed

Requirement	Implementation
Exactly two inputs	The executable requires a course file and a faculty file. Any other argument count is rejected.
Modular C design	Input, scheduling, output, and main-control responsibilities are separated.
Static library	Reusable modules are archived in lib/libschedule.a and linked to the main object.
Makefile	Rules compile, archive, link, run, clean, rebuild, and generate the Word output.
Sample output	The build includes generated CSV, text, and Word schedule files.

2 Software Design

The design uses one public header and four C source files. The main module coordinates the workflow but does not perform parsing, scheduling, or formatting itself. This separation makes each responsibility easier to understand, test, and replace.

File	Responsibility
include/scheduler.h	Shared structures, limits, and public function prototypes.
src/input.c	Reads pipe-delimited records, trims fields, parses days and times, and reports malformed data.
src/scheduling.c	Checks lecturer and class conflicts, chooses office-hour blocks, and associates each block with the neares
src/output.c	Writes class_schedule.csv, office_hours.csv, and schedule.txt.
src/main.c	Requires exactly two file arguments and runs the modules in order.
scripts/generate_docx.py	Reads only the C program's CSV output and creates the formatted Word document.

Program flow

1. Open and parse the course file.
2. Open and parse the faculty file and its office-hour availability table.

3. Verify that each course has a known lecturer and that a lecturer has no overlapping classes.
4. For each professor, examine availability in file order and skip any block that overlaps a class.
5. Select enough minutes to satisfy the required total and trim only the final block if necessary.
6. Write the generated schedules and optionally create the Word document.

3 Static Library Research and Use

A static library is an archive of compiled object files. GNU's ar documentation explains that an archive stores a collection of member files and is commonly used as a library of reusable subroutines. The s option creates or updates a symbol index, which makes linking faster. In this project, input.o, scheduling.o, and output.o are the archive members.

The library is created with:

```
ar rcs lib/libschedule.a build/input.o build/scheduling.o build/output.o
```

The r option inserts or replaces members, c creates the archive when needed, and s writes the symbol index. The result follows the conventional name libschedule.a. GCC documents that static libraries are archives of object files named libNAME.a and that -lNAME searches for the named library. The Makefile therefore links with:

```
gcc build/main.o -Llib -lschedule -o bin/schedule_generator
```

The -Llib option adds the local lib directory to the search path, while -lschedule selects libschedule.a. The main object appears before the library because GNU linkers process files in command order. This statically copies the required project-module code into the executable. It does not claim that every operating-system library in the final executable is statically linked.

Why a library is appropriate

The library creates a clear boundary between the reusable scheduling services and the application entry point. Another command-line interface or graphical interface could call the same functions without copying the parsing and scheduling code. It also demonstrates separate compilation: changing main.c does not require recompiling the library modules.

4 Makefile Automation

A Makefile expresses targets, prerequisites, and recipes. Make compares file timestamps and rebuilds a target when a prerequisite is newer or the target does not exist. Header dependencies are included on every object rule, so a change to scheduler.h causes the affected C files to compile again. This reduces repeated typing and makes the build reproducible.

The default all target builds the executable. Pattern rules compile every src/name.c file into build/name.o. The archive rule creates libschedule.a from the reusable objects, and the executable rule links main.o with that archive. The run target invokes the program with exactly two input files. The documents target first runs the C program and then calls Python to convert the generated CSV tables into Faculty_Schedule.docx. The clean target removes only known generated artifacts.

Typical use

```
make
make run
make documents
make clean
```

5 Input Files and Access Decision

The two input files are intended to be stored locally on the laboratory computer inside the project's input directory. I selected local files because the assignment requires exactly two inputs, lab Internet access may be unavailable, and a local copy makes the same test repeatable. The program has no network dependency and does not download or scrape private faculty information. If a registrar or department website is the original source, its information should first be reviewed and saved into the documented local format.

At run time, main passes both command-line paths to the library. input.c calls fopen in read mode, processes one line at a time with fgets, separates fields at pipe characters, and closes each stream. A pipe delimiter was selected because commas are valid inside day lists and course lists. Lines beginning with # are comments.

Course file record

```
course code | title | lecturer | days | start | end | room
```

Faculty file record

```
lecturer | department | office | required hours | availability | courses taught
```

Times use 24-hour HH:MM values. Availability uses Day=HH:MM-HH:MM, semicolons between days, and commas between multiple blocks on one day. Lecturer names must match exactly between the files. The supplied sample records list Dr. Appolo Tankeh and retain the courses, times, rooms, and office hours shown in the assignment example.

6 Scheduling Rules and Error Handling

The faculty file gives the professor's required weekly office-hour total and acceptable time blocks. For every professor, the generator considers these blocks in the order entered. A block is ignored if it overlaps one of that professor's classes. Accepted minutes are accumulated until the requirement is reached. If the final block is longer than the remaining requirement, its end time is shortened. If all usable availability is exhausted first, the program stops with a specific error instead of silently producing an incomplete schedule.

Each selected block is labeled with the nearest class taught by the same lecturer on the same day. A block after COSC 330 therefore receives the COSC 330 label, while a block immediately before COSC 631 receives COSC 631. If the professor teaches no course that day, the block is labeled General. The label is informational and does not prevent students from other courses from attending.

Check	Response
Missing file	Report which path could not be opened and exit with failure.
Wrong field count	Report the input line number.

Check	Response
Invalid day or time	Reject the record before scheduling.
Unknown lecturer	Identify the course and unmatched lecturer.
Overlapping classes	Identify both course codes and the lecturer.
Insufficient availability	Report how many additional office-hour hours are needed.

7 Output and Verification

The C program writes class_schedule.csv, office_hours.csv, and schedule.txt. The CSV files preserve a simple structure that can be opened in spreadsheet software or passed to another formatter. The text file gives a readable terminal version. The optional Python script consumes those generated CSV files and creates a Word table resembling the supplied example. This keeps scheduling decisions in C and presentation formatting in Python.

Generated sample office hours

Day	Office hours	Related course
Monday	10:45 AM - 1:00 PM	COSC 330
Tuesday	10:45 AM - 1:00 PM	COSC 369
Wednesday	10:45 AM - 1:00 PM	COSC 330
Wednesday	3:00 PM - 5:00 PM	COSC 631

Verification performed

- The project was rebuilt from a clean state with C11 warnings enabled.
- The archive contents were checked to confirm that input.o, scheduling.o, and output.o are members of libschedule.a.
- The executable processed the two sample files and generated four office-hour blocks.
- The resulting times were compared with the supplied sample image.
- The Word schedule and this PDF report were rendered to page images and visually inspected for clipping and overlap.

8 Limitations and Possible Improvements

The current version uses fixed maximum record counts and requires exact lecturer-name matches. It assumes that the faculty member supplies valid availability in priority order and does not optimize a department-wide objective such as minimizing gaps or distributing hours across every weekday. A later version could use dynamic memory, case-insensitive identifiers, formal unit tests, and an optimization method that scores alternative schedules. Those improvements are outside the first project's scope but the

module boundaries allow them to be added without rewriting main.c.

9 Conclusion

The system meets the core assignment goals: it reads exactly two local files, generates teaching and office-hour schedules from data rather than hard-coded tables, separates responsibilities into C modules, packages reusable modules in a static library, and automates compilation and document creation through a Makefile. The included README gives the commands and data format needed to reproduce the result on a laboratory computer.

References

Free Software Foundation. (n.d.). GNU ar. GNU Binary Utilities. https://sourceware.org/binutils/docs/binutils/ar.html

Free Software Foundation. (n.d.). GNU make manual. https://www.gnu.org/software/make/manual/

GNU Project. (n.d.). Options for linking. Using the GNU Compiler Collection. https://gcc.gnu.org/onlinedocs/gcc/Link-Options.html